

Практика в ЦБС Петроградского района

Подготовили: Лелло Андрей, Габайдулин Святослав, Махов Святослав
Группа 206_ПОО/24

Часть 1. Анализ применимости в наших проектах

1.1 Описание наших существующих проектов

Наши группы разрабатывают и сопровождают следующие проекты:

nginx-proxy / vbkg.pr-cbs.ru	Nginx-прокси-сервер, предположительно обслуживающий один или несколько веб-ресурсов (vbkg.pr-cbs.ru). Принимает входящие запросы, маршрутизирует по доменам/путям, обеспечивает SSL.
Боты библиотек	Telegram-боты для библиотечных сервисов: уведомления о задолженностях, поиск книг, информирование читателей.

1.2 Где и как ML-сервис можно применить

После анализа функционала prog-6-ml-service (загрузка модели, предсказание по JSON/CSV, healthcheck) определены следующие точки интеграции:

Интеграция в бота библиотеки

Наиболее органичная точка применения — Telegram-боты библиотеки. ML-сервис может добавить следующие интеллектуальные возможности:

- Рекомендации книг на основе истории читателя. Модель обучается на данных о читательских предпочтениях и выдаёт персонализированные рекомендации. Бот вызывает /predict с историей читателя и получает список рекомендованных книг.
- Классификация запросов пользователей. Модель NLP определяет тип запроса (поиск книги, уточнение задолженности, запрос расписания) и направляет пользователя в нужный сценарий бота.
- Предсказание срока возврата. Модель предсказывает вероятность просрочки возврата книги на основе паттернов читателя, что позволяет боту отправлять упреждающие напоминания.

Отчёт рабочей группы №1 Задание 2

- Пакетная обработка через `/predict-from-csv`. Библиотека может загружать список читателей в CSV-формате и получать предсказания по всей базе для массовой рассылки.

Схема взаимодействия: бот отправляет HTTP-запрос на `/predict` ML-сервиса, сервис возвращает предсказание, бот формирует ответ пользователю на основе результата.

.

1.3 Выбор целевого проекта

Рекомендуется внедрять ML-сервис в проект Telegram-бота библиотеки, поскольку:

- Бот уже имеет Python-стек, что упрощает HTTP-взаимодействие с FastAPI/Flask бэкендом.
- Функционал `/predict` и `/predict-from-csv` напрямую применим для задач рекомендации и классификации запросов.
- Минимальные изменения в инфраструктуре: ML-сервис запускается как дополнительный Docker-контейнер рядом с ботом.
- CI/CD уже настроен в исходном репозитории — можно переиспользовать подход с Container Registry.

Часть 2. Roadmap внедрения ML-сервиса в бота библиотеки

План работ по интеграции функциональности из репозитория prog-6-ml-service в проект Telegram-бота библиотеки. Roadmap разбит на 5 фаз.

2.1 Сводная таблица фаз

Фаза	Срок	Задачи	Результат
Фаза 1 Подготовка	1–2 недели	Аудит существующего бота, определение ML-задачи, сбор и анализ данных для обучения модели.	Техническое задание на интеграцию, датасет.
Фаза 2 Модель	2–3 недели	Выбор и обучение ML-модели, экспорт в .pkl/.joblib, тестирование через /predict вручную.	Готовая модель, проверенная через API.
Фаза 3 Инфраструктура	1 неделя	Добавление ML-сервиса в docker-compose бота, настройка сети, адаптация .gitlab-ci.yml.	ML-сервис запущен рядом с ботом.
Фаза 4 Интеграция	2 недели	Написание клиентского кода в боте для вызова /predict, обработка ответов, UX-сценарии.	Бот использует ML-рекомендации в диалоге.
Фаза 5 Тест и деплой	1 неделя	Тестирование end-to-end, нагрузочное тестирование, деплой на продакшн через CI/CD.	Фича в продакшн, мониторинг активен.

2.2 Детализация по фазам

Фаза 1 — Подготовка и анализ (1–2 недели)

1. Провести аудит существующего Telegram-бота: изучить текущую архитектуру, стек технологий, способ деплоя.
2. Определить конкретную ML-задачу: рекомендации книг, классификация запросов пользователей или предсказание просрочки.
3. Выгрузить исторические данные из БД библиотеки: история выдач, читательские профили, запросы в боте.
4. Провести разведочный анализ данных (EDA): оценить качество данных, выявить пропуски, определить признаки.
5. Составить ТЗ на интеграцию и согласовать с командой.

Фаза 2 — Обучение ML-модели (2–3 недели)

6. Выбрать алгоритм ML в зависимости от задачи: collaborative filtering (рекомендации), логистическая регрессия или BERT (классификация запросов).
7. Разделить данные на train/test, провести обучение модели в Jupyter Notebook.
8. Оценить качество модели (accuracy, F1, NDCG в зависимости от задачи), при необходимости — тюнинг гиперпараметров.
9. Экспортировать модель через `joblib.dump()` или `pickle.dump()` в файл `.pkl/.joblib`.
10. Загрузить модель на сервер через POST `/upload-model` и проверить работу `/predict` с тестовыми данными.
11. Задokumentировать формат входных данных для `/predict` (JSON-схему).

Фаза 3 — Инфраструктурная интеграция (1 неделя)

12. Склонировать репозиторий `prog-6-ml-service` или скопировать `backend/` в репозиторий бота как подмодуль или отдельный сервис.
13. Дополнить `docker-compose` бота новым сервисом `ml-backend` на основе образа из Container Registry ML-сервиса.
14. Настроить внутреннюю Docker-сеть: бот и ML-сервис должны видеть друг друга по имени сервиса (например, `http://ml-backend:8000`).
15. Добавить переменную окружения `ML_SERVICE_URL` в конфигурацию бота.
16. Расширить `.gitlab-ci.yml` бота: добавить стадию деплоя ML-сервиса и `healthcheck /status` перед запуском основного деплоя.
17. Проверить работу всего стека локально через `docker-compose up`.

Фаза 4 — Разработка интеграционного кода (2 недели)

18. Написать Python-клиент для ML-сервиса в коде бота: функции `call_predict(data)` и `call_predict_csv(file_path)` на базе библиотеки `httpx` или `requests`.
19. Реализовать UX-сценарий рекомендаций: новая команда `/recommend` или inline-кнопка в диалоге, которая вызывает ML-сервис и отображает результат.
20. Добавить обработку ошибок: если ML-сервис недоступен (таймаут, 5xx), бот корректно сообщает пользователю об ошибке и продолжает работу без рекомендаций.
21. Реализовать кэширование результатов: если один и тот же читатель запрашивает рекомендации дважды за короткое время, использовать кэш (Redis или in-memory).
22. Написать unit-тесты для нового кода бота с mock-ответами от ML-сервиса.

Фаза 5 — Тестирование и деплой (1 неделя)

23. Провести end-to-end тестирование: полный сценарий от запроса пользователя в Telegram до ответа с рекомендацией.
24. Выполнить нагрузочное тестирование ML-сервиса (locust или k6): убедиться, что /predict выдерживает ожидаемый RPS.
25. Настроить мониторинг: добавить /status ML-сервиса в систему мониторинга (например, Uptime Kuma или healthcheck в docker-compose с restart: always).
26. Задеплоить на продакшн через GitLab CI/CD. Убедиться, что пайплайн проходит все стадии (lint, build, test, deploy).
27. Написать документацию для команды: как обновить модель (через /upload-model), как откатить на предыдущую версию.

2.3 Риски и способы их митигации

Качество данных	Исторических данных может быть недостаточно для обучения. Митигация: начать с простой модели (popularity-based recommendations), постепенно усложняя.
Производительность ML-сервиса	Предсказания могут быть медленными. Митигация: кэширование результатов, установка таймаута 2–3 секунды в боте.
Падение ML-сервиса	Бот должен работать без ML. Митигация: graceful degradation — при недоступности сервиса бот работает в штатном режиме без рекомендаций.
Дрейф модели	Со временем качество предсказаний снижается. Митигация: периодическое переобучение модели (раз в месяц) с загрузкой через /upload-model.
Безопасность API	ML-сервис не должен быть доступен публично. Митигация: сервис работает только во внутренней Docker-сети, без проброса порта наружу.

2.4 Итоговые метрики успеха

- ML-сервис отвечает на /predict за время менее 500 мс в 95% случаев.
- Бот корректно обрабатывает недоступность ML-сервиса (fallback без ошибок пользователю).
- Пайплайн CI/CD проходит все стадии без ручного вмешательства.
- Рекомендации релевантны: не менее 30% пользователей кликают на рекомендованные книги (CTR).
- Документация позволяет любому члену команды обновить модель самостоятельно.